HOCHSCHULE
RAVENSBURG-WEINGARTEN
UNIVERSITY
OF APPLIED SCIENCES

Grundpraktikum 3
Mikrocontroller

Task 3

Controlling PWM using a pushbutton with external Interrupt

Student 1 : Leandro Ebner (Matrikel-Nr: 14740284)

Student 2 : Caleb Patrick DSouza (Matrikel-Nr: 28840332)

Winter Semester 2023

Page Contents

I. INTRODUCTION 3
II. BASIC THEORY 4
1. Arduino Nano 4
2. Push Button 5
3. LED 5
4. Debounce Effect 6
5. PWM 6
6. INTERRUPT 7
III. HARDWARE DESIGN AND DEVELOPMENT 8
1. Block Diagram 8
2. Schematic 8
3. Hardware Setup 8
IV. SOFTWARE DESIGN AND DEVELOPMENT 9
1. Flowchart 9
2. Functions 9
a. Initialization Variable 9
b. Setup Function 10
c. Interrupt Function 11
d. Heart Beat with the Timer Function 12
e. PWM Function 12
f. Debounce Function 13
g. Software Source Code 14
V. RESULT 15

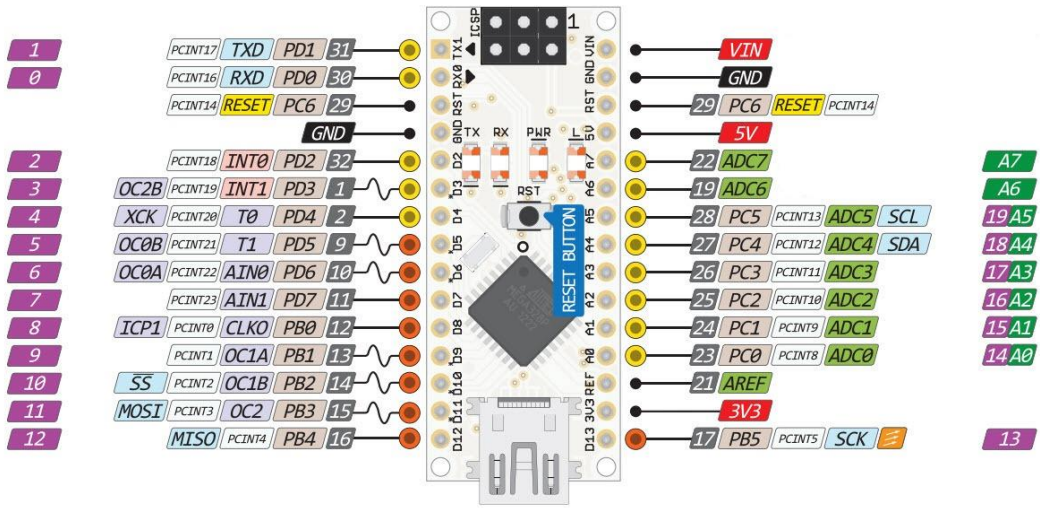
I. INTRODUCTION

The aim of the “Task 3. Controlling PWM using a pushbutton with external interrupt” is to make the participants able to do the following task below:

1. Create the Hardware Setup
2. Implement the Interrupt Function for the Push Button
3. Implement the Pulse Width Modulation Function
4. Implement the Debounce Function

II. BASIC THEORY

1. Arduino NANO



The Arduino Nano is a compact microcontroller board with a small computer known as a microcontroller (ATmega328P). It features pins for connecting and controlling external devices, including digital and analog pins. You can power it through a USB cable or an external power source, thanks to a built-in voltage regulator. The USB interface lets you program it and communicate with it.

The microcontroller runs at a clock speed of 16 MHz, and you can write and upload code to it using the Arduino IDE, which uses the C/C++ programming language. The board comes with built-in LEDs for simple status indications and supports various communication protocols like UART, I2C, and SPI. It's part of the open-source Arduino platform, allowing for community collaboration and sharing of designs and ideas.

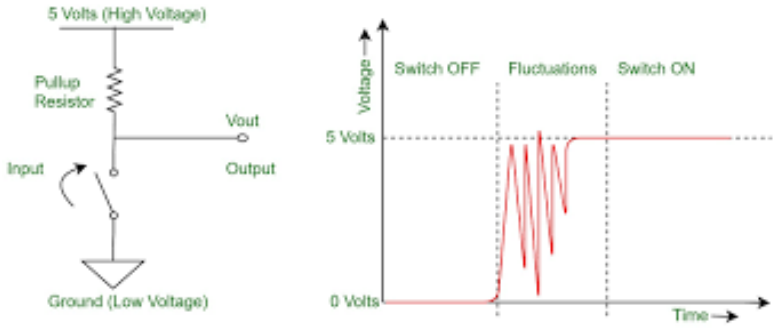
2. Push Button

A push button works with an Arduino Nano by connecting one side of the button to a digital input pin on the Arduino and the other side to ground (GND). When you press the button, it creates a connection between the digital pin and ground, which the Arduino can detect. You can then write code in the Arduino IDE to respond to this button press, like turning on an LED or executing a specific action when the button is pressed. So, the button serves as a way for you to interact with the Arduino and trigger actions in your projects.

3. LED

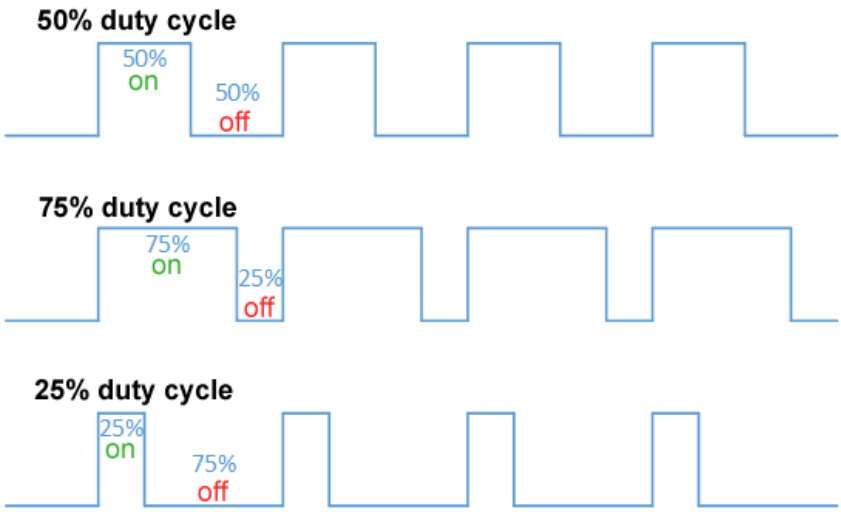
An LED (Light Emitting Diode) is a tiny light that can be turned on and off. In an Arduino Nano, you can connect an LED to one of its pins. By writing code, you can tell the Arduino to turn the LED on or off. This is handy for creating visual indicators, like making a light blink when something happens, or for illuminating your projects. So, an LED is like a controllable light you can use with an Arduino Nano to add visual effects to your creations.

4. Debounce Effect



The debounce effect occurs in electronic switches and sensors when mechanical bouncing causes rapid and unintended fluctuations in the signal. Debouncing is the process of stabilizing this signal to ensure accurate interpretation of a single, clean input. It is essential in digital systems to prevent unintended triggering or false readings. Debouncing can be achieved through electronic circuits or software algorithms. In software, debounce algorithms filter out noise during the bouncing period, allowing only stable state changes to be recognized. This is crucial in applications like button presses and digital sensors. The goal is to improve the reliability and accuracy of input detection by eliminating unintended signals caused by mechanical bouncing.

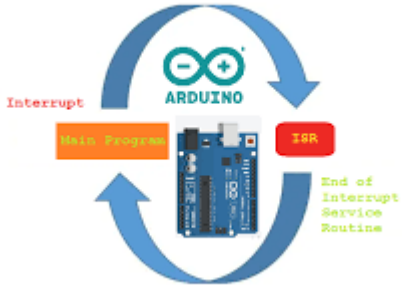
5. Pulse Width Modulation (PWM)



Pulse Width Modulation (PWM) is a technique used in electronics to control the average power delivered to a load. It involves rapidly switching a signal between high and low states at a fixed frequency. The percentage of time the signal is in the high state, known as the duty

cycle, determines the average power. PWM is commonly employed to control the speed of motors, regulate LED brightness, and modulate audio signals. The higher the duty cycle, the more power is delivered, and vice versa. PWM provides an efficient way to control analog devices using digital signals. It is widely used in microcontroller-based systems for its simplicity and effectiveness in achieving variable outputs with a fixed voltage level. The smooth transitions between high and low states allow for precise control and modulation in various electronic applications.

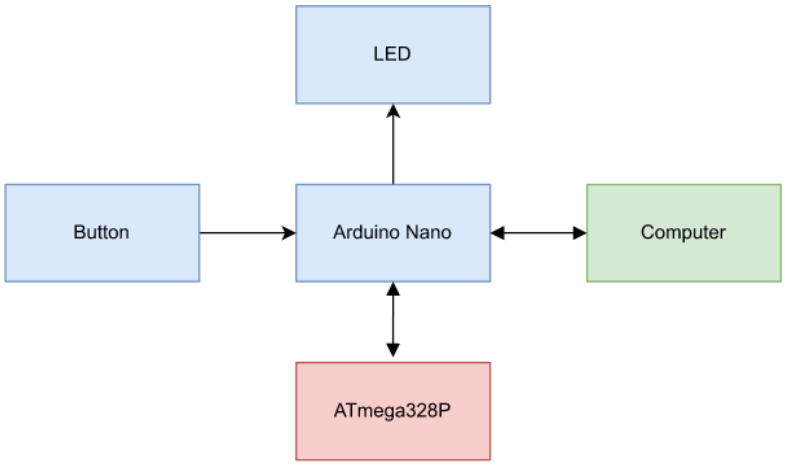
6. Interrupt



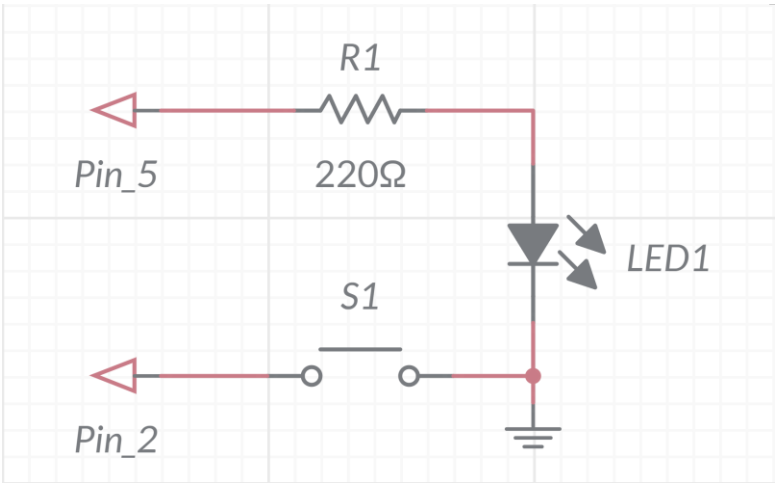
Interrupts in microcontroller programming allow the processor to respond to external events without constant checking. When an interrupt occurs, the processor pauses its current execution and jumps to a specific Interrupt Service Routine (ISR) to handle the event. This ISR performs the necessary tasks related to the triggered event. After completing the ISR, the processor returns to the main program, continuing its execution as if the interrupt never occurred. Interrupts are essential for managing time-sensitive or asynchronous events efficiently, providing a more responsive way to handle external stimuli. They allow the microcontroller to perform other tasks while waiting for interrupts, avoiding the need for constant polling.

III. HARDWARE DESIGN AND DEVELOPMENT

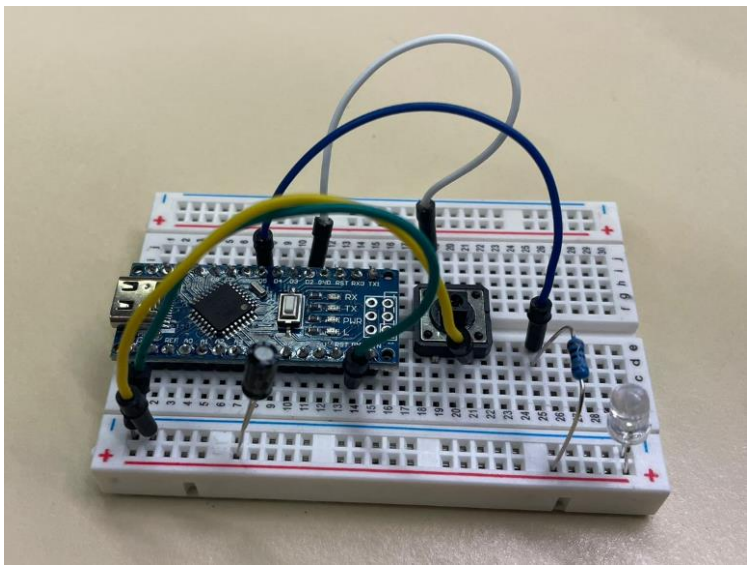
1. Block Diagram



2. Schematic

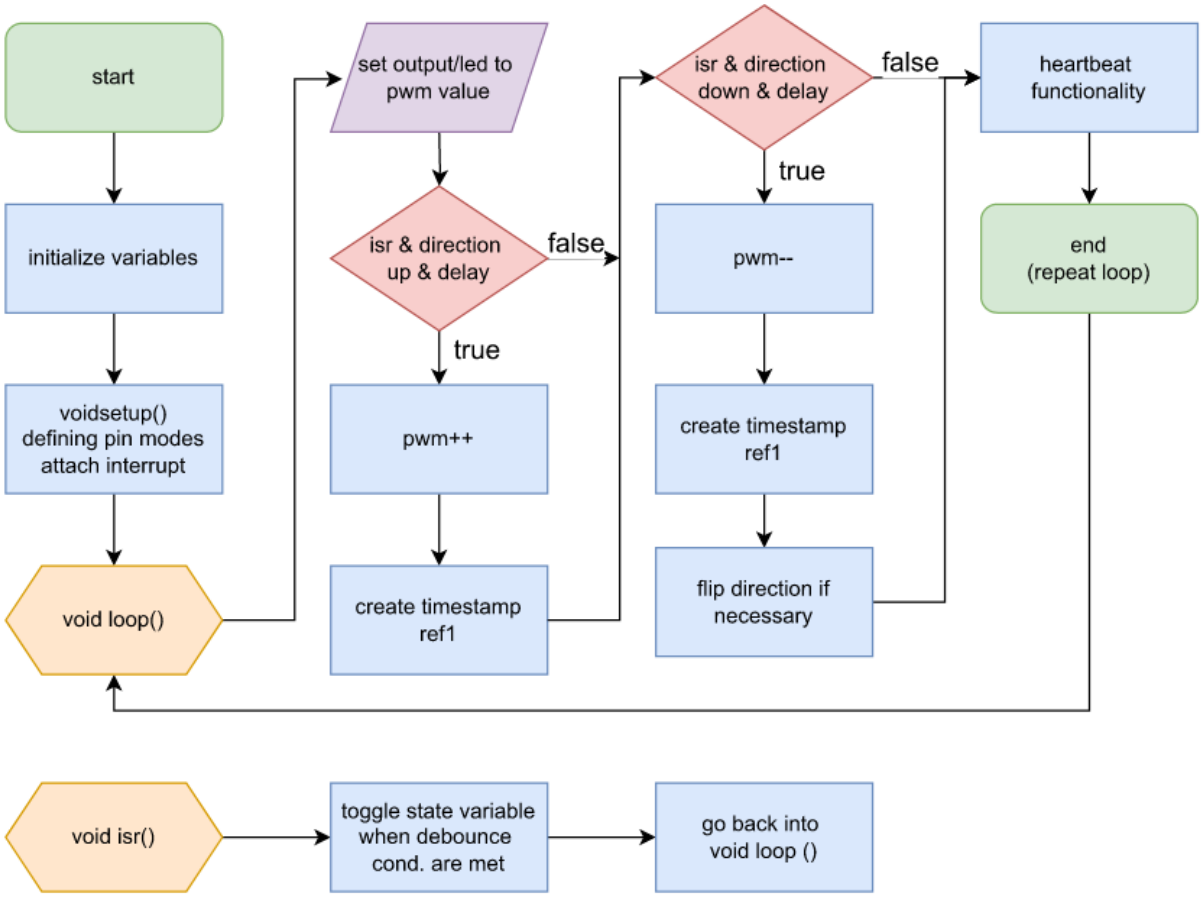


3. Hardware Setup



IV. SOFTWARE DESIGN AND DEVELOPMENT

1. Flowchart



2. Functions

a. Initialization Variable

```
int dir;
int pwm;
int state;
int sample;
const int led = 5;
const int button = 2;
const int inbuilt_led = 13;
unsigned long ref1;
unsigned long ref2;
unsigned long ref3;
```

The code initializes variables to control PWM brightness, manage the main state, and handle timing aspects. Key variables include:

- `dir`: Manages brightness ramping direction.
- `pwm`: Represents PWM brightness (0 to 255).
- `state`: Main toggle for adjusting LED brightness.
- `sample`: Sample input from the pushbutton (not actively used).
- `led`: Digital pin for the PWM-controlled LED (pin 5).
- `button`: Digital pin for the pushbutton (pin 2).
- `inbuilt_led`: Digital pin for the built-in LED (pin 13).
- `ref1`, `ref2`, `ref3`: Time stamp variables for timing control.

These variables facilitate PWM LED control, toggle state changes based on pushbutton input, and manage timing-related functions like heartbeat generation and debounce control. The values are dynamically updated during program execution.

b. Setup Function

```
void setup() {

pinMode(led, OUTPUT);
pinMode(inbuilt_led, OUTPUT);
pinMode(button, INPUT_PULLUP);
attachInterrupt(digitalPinToInterrupt(button), isr, FALLING);

}
```

The setup function initializes the hardware configuration for the microcontroller-based system. In a scientific context:

- pinMode(led, OUTPUT): Configures the LED pin (digital pin 5) as an output, allowing control of the LED's brightness through PWM.

- pinMode(inbuilt_led, OUTPUT): Configures the built-in LED pin (digital pin 13) as an output, facilitating control for indicating system status.

- pinMode(button, INPUT_PULLUP): Configures the button pin (digital pin 2) as an input with a pull-up resistor, ensuring stable reading of button state.

- attachInterrupt(digitalPinToInterrupt(button), isr, FALLING): Sets up an external interrupt on the button pin, linking it to the Interrupt Service Routine (ISR) named isr and triggering on a falling edge (button press).

c. Interrupt Function

```
void isr() {


if(millis() - ref3 > 100){state = !state;}

}
```

The isr function, an Interrupt Service Routine, is triggered by an external interrupt, likely associated with a pushbutton press. In this concise scientific explanation:

- Toggle Mechanism: The function toggles the state variable between active and inactive states.

- Debouncing: It includes a debouncing condition (millis() - ref3 > 100) to ensure stable state toggling and mitigate undesired effects caused by mechanical contacts.

This interrupt function is crucial for responsive handling of pushbutton input, facilitating reliable toggling of the system's main state.

d. Heart Beat with the Timer Function

```
if(millis() - ref2 >= 500){
  digitalWrite(inbuilt_led, !digitalRead(inbuilt_led));
  ref2 = millis();}

}
```

The heartbeat function in the provided code toggles the built-in LED at pin 13 at a regular interval, creating a pulsating effect indicative of system activity. Scientifically:

- Frequency Control: The built-in LED toggles its state every 500 milliseconds (millis() - ref2 >= 500), adjusting the heartbeat signal's frequency.

- Toggle Mechanism: Utilizes the digitalWrite function to toggle the state of the built-in LED, creating a visual indicator of the system's operational status.

e. PWM Function

```
void loop() {

analogWrite(led, pwm);

if(state == 1 && dir == 0 && millis() - ref1 >= 5){
  pwm++;
  ref1 = millis();}

if(state  == 1 && dir == 1 && millis() - ref1 >= 5){
  pwm--;
  ref1 = millis();}

if(pwm == 0)
    {dir = 0;

    }
if(pwm == 255)
    {dir = 1;

    }
```

This code, within the loop function, orchestrates the dynamic control of LED brightness through Pulse Width Modulation (PWM) while considering system state, ramping direction, and specified timing conditions. Scientifically:

- PWM Control: Utilizes analogWrite to adjust the LED's brightness based on the current pwm value.

- Ramping Mechanism: If the system is in an active state (state == 1), it incrementally adjusts pwm upwards (dir == 0) or downwards (dir == 1) with a specified interval of 5 milliseconds (millis() - ref1 >= 5).

- Direction Control: Checks for pwm reaching minimum (0) or maximum (255) values, toggling the dir variable to indicate the direction for cycling through brightness levels.

f. Debounce Function

We do not need a Debounce function and below explains the reason here:

`FALLING` edge triggering captures the moment a button is pressed, but it doesn't eliminate the bouncing effect. If the button bounces during the falling edge, it may lead to multiple triggering. In contrast, `CHANGE` triggering responds to both rising and falling edges, potentially causing more transitions. Therefore, `CHANGE` often requires a debounce function to ensure stable and accurate detection of button presses or releases.

g. Software Source Code

```
int dir;                                                    //variable for brightness ramping direction
int pwm;                                                    //variable for pwm brightness
int state;                                                  //main virtual toggle variable
int sample;                                                 //sample input of button
const int led = 5;                                          //maps led pin
const int button = 2;                                       //maps button pin
const int inbuilt_led = 13;                                 //maps inbuilt led pin
unsigned long ref1;                                         //time stamp for ramping speed
unsigned long ref2;                                         //time stamp for heartbeat
unsigned long ref3;                                         //time stamp for debouncing the button

void setup() {

pinMode(led, OUTPUT);                                       //defines led as an output
pinMode(inbuilt_led, OUTPUT);                               //defines inbuilt led as an output
pinMode(button, INPUT_PULLUP);                              //defines button as input
attachInterrupt(digitalPinToInterrupt(button), isr, FALLING);   //set up interrupt routine, pin and mode

}

void loop() {

analogWrite(led, pwm);

if(state == 1 && dir == 0 && millis() - ref1 >= 5){         //ramping up the brightness of the led
  pwm++;
  ref1 = millis();}

if(state  == 1 && dir == 1 && millis() - ref1 >= 5){        //ramping down the brightness of the led
  pwm--;
  ref1 = millis();}

if(pwm == 0)
    {dir = 0;                                               //checks direction to cycle through the brightness levels

    }
if(pwm == 255)
    {dir = 1;                                               //change direction as soon as the led is at full brightness

    }


if(millis() - ref2 >= 500){                                 //implements the heartbeat functionality
  digitalWrite(inbuilt_led, !digitalRead(inbuilt_led));
  ref2 = millis();}

}

void isr() {


if(millis() - ref3 > 100){state = !state;}                  //toggle main variable and check for debounce conditions

}
```

V. RESULT

- The implemented hardware setup demonstrated stable connections and proper functionality, ensuring reliable interaction between the pushbutton and microcontroller.
- The Interrupt Service Routine successfully detected and responded to button presses, facilitating a seamless transition between PWM control states.
- The PWM function enabled precise modulation of the output signal, showcasing varying levels of intensity corresponding to different duty cycles.
- The incorporated debounce function effectively mitigated button bounce, contributing to the overall robustness and accuracy of the system.